

Justificacion del uso de SQL (PostgreSQL / Supabase)

Trabajo Practico - LinkPro (clon de LinkedIn)

1. Contexto del trabajo practico

El TP consiste en construir una red social similar a LinkedIn integrando multiples motores de bases de datos para distintas funcionalidades. La consigna exige usar al menos una base de datos relacional (SQL), una documental (MongoDB) y una de grafos o clave-valor. La eleccion de que tecnologia usar para cada modulo debe estar justificada por las características propias de los datos y las operaciones que se realizan sobre ellos.

2. Funcionalidades implementadas sobre SQL

Las siguientes funcionalidades del sistema se apoyan en tablas SQL gestionadas a traves de Supabase (PostgreSQL):

Modulo	Tablas involucradas
Usuarios y autenticacion	users, roles, user_roles, password_reset_tokens
Perfil profesional	users, work_experience, education, skills, user_skill
Empresas	companies
Ofertas de empleo	jobs, job_skill
Postulaciones	applications

3. Por que SQL para estas funcionalidades

3.1 Integridad referencial y relaciones fuertes

Los datos de usuarios, perfiles, empleos y postulaciones estan fuertemente relacionados entre si. Las claves foraneas de SQL garantizan que no existan postulaciones huérfanas, ni ofertas que referencien usuarios o empresas inexistentes. Esto seria mucho mas dificil de garantizar en una base de datos documental sin logica de aplicacion adicional.

- Una oferta de empleo (jobs) pertenece a un usuario (poster_user_id -> users.user_id) y opcionalmente a una empresa (company_id -> companies).
- Una postulacion (applications) vincula un usuario con una oferta (user_id -> users, job_id -> jobs).
- Las habilidades del usuario (user_skill) y las requeridas por una oferta (job_skill) referencian el catalogo comun skills.

3.2 Consultas relacionales complejas en una sola operacion

La API utiliza las capacidades de join de Supabase PostgREST para traer datos de multiples tablas en una sola consulta. Por ejemplo, al listar ofertas:

```
jobs
+-- companies(name)
+-- job_skill
```

```
+-- skills(skill_id, name, type)
```

Y al listar postulaciones de un usuario:

```
applications
+-- jobs
|   +-- companies(name)
+-- users(user_id, name, surname, email, profile_photo_url)
```

Con documentos MongoDB habria que hacer multiples consultas o disenar documentos embebidos que no representan bien la realidad del modelo, ya que las mismas skills son compartidas por usuarios y ofertas, y las mismas companies son compartidas por experiencias laborales y ofertas.

3.3 Normalizacion: evitar duplicacion de datos

El catalogo de habilidades (skills) es un claro ejemplo de por que la normalizacion relacional es conveniente:

- Una habilidad como "Python" existe una sola vez en la tabla skills.
- Tanto el perfil de un usuario (user_skill) como una oferta de empleo (job_skill) la referencian mediante su skill_id.
- Si el nombre de una habilidad cambiara, se actualiza en un unico lugar.

Con un modelo documental, "Python" apareceria embebida en el documento de cada usuario y en el de cada oferta, generando inconsistencias.

3.4 Unicidad y restricciones de negocio

SQL permite declarar restricciones directamente en el esquema, garantizandolas en la capa de almacenamiento:

- users.email es UNIQUE: no pueden existir dos cuentas con el mismo email.
- users.dni es UNIQUE: no puede haber duplicados de documento.
- applications(job_id, user_id) es unico: un usuario no puede postularse dos veces a la misma oferta.
- Campos como is_active, status e is_current tienen valores por defecto y restricciones de tipo.

3.5 Transacciones ACID

Las operaciones criticas del sistema requieren atomicidad:

- Al registrar un usuario: se inserta en users y se asignan roles en user_roles. Si una operacion falla, la cuenta no queda en estado inconsistente.
- Al publicar una oferta con habilidades: se inserta en jobs y luego en job_skill de forma atomica.

Las garantias ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) de PostgreSQL aseguran consistencia incluso bajo fallas parciales o concurrencia de multiples usuarios.

3.6 Esquema predecible para datos estructurados

Los datos de usuarios, perfiles y empleos tienen una estructura fija y bien definida: todos los usuarios tienen nombre, email y contrasena; todas las ofertas tienen titulo, modalidad y estado. Esta homogeneidad favorece el modelo relacional. Un motor documental como MongoDB es mas conveniente cuando los datos son heterogeneos o evolucionan rapidamente en estructura (por ejemplo, el contenido de posts o comentarios).

4. Diagrama de relaciones (simplificado)

```
users ----- user_roles ---- roles
|
+--- work experience ---- companies
+--- education
+--- user_skill ----- skills
|
+--- jobs (poster_user_id) --- companies
|
+--- job_skill -----++
|
+--- applications ---- users
```

5. Integracion en la arquitectura del sistema

Backend: Flask + Supabase Python SDK

El backend expone una API REST construida con Flask, organizada en blueprints por dominio:

- /auth -> auth_bp: registro, login, recuperacion de contraseña.
- /profile -> profile_bp: datos del perfil, foto, experiencia, educacion, habilidades.
- /jobs -> jobs_bp: listado de ofertas, publicacion, postulaciones, gestion de candidatos.

Cada blueprint interactua con Supabase mediante el cliente supabase_admin (clave de servicio), que ejecuta queries sobre PostgreSQL usando la API PostgREST.

Supabase como infraestructura SQL

- PostgreSQL gestionado: base de datos relacional con todas las garantias de consistencia.
- PostgREST: convierte automaticamente las tablas en endpoints REST con soporte de filtros, joins y paginacion.
- Storage: almacenamiento de archivos (fotos de perfil) integrado con el mismo proyecto.
- Row Level Security (RLS): politicas de acceso a nivel de fila. En el TP se usa la clave service_role desde el backend.

6. Comparacion con las otras bases del proyecto

Criterio	SQL (PostgreSQL)	MongoDB (documental)	Grafo / Clave-valor
Estructura	Fija, tabular	Flexible, anidada	Relaciones / pares
Relaciones	Claves foraneas, JOINS	Referencias manuales	Aristas del grafo
Caso de uso TP	Usuarios, perfiles, empleos	Posts, comentarios, grupos	Conexiones entre usuarios
Justificacion	Datos con muchas relaciones	Contenido heterogeneo	Traversals de red social

7. Conclusion

La eleccion de SQL para los modulos de usuarios, perfiles, empleos y postulaciones esta justificada por:

- La naturaleza relacional de los datos: usuarios, empresas, ofertas y postulaciones se interconectan mediante referencias que deben ser consistentes.

Justificacion del uso de SQL en el TP - LinkPro

- La necesidad de integridad referencial: las claves foraneas garantizan que los datos no queden en estados invalidos.
- La eficiencia de las consultas con JOINS: se obtienen datos de multiples tablas en una unica operacion.
- La normalizacion: skills y companies son entidades compartidas que no deben duplicarse.
- Las restricciones de unicidad: email, DNI y postulaciones unicas por usuario/oferta son propiedades del esquema.
- Las garantias ACID: operaciones criticas como el registro de usuarios o la publicacion de ofertas son atomicas.

Estas características hacen de PostgreSQL/Supabase la tecnología adecuada para el núcleo transaccional del sistema, mientras que MongoDB y otras bases complementan el sistema para datos menos estructurados como publicaciones, comentarios y conexiones sociales.